DAY 1

Problem Solving Techniques and Data Structures I

Set 1–3 Practical Record

E-Commerce Catalog and Traffic Analysis • Insurance Claims Processing • Logistics Network and Delivery Windows

Student details

Field	Fill in before submission
Student name	Kuchi Yasaswini
Roll number	-
Class / batch	Batch 1
Date	30-10-2026

Aim

Answer the fifteen assigned questions with Python, concrete dry runs, complexity notes, and console captures from executed programs.

Use of data

Public source data is used wherever the required fields are available. Any graph or event field not present in public records is explicitly marked as a modeled test or proxy; no invented test event is described as an observed record.

What is included

- Set overview pages with the business scenario, tasks, and deliverables.
- A ten-row real Olist sample, field mapping, code, dry run, and one actual terminal screenshot for each question.
- A decision/complexity table and source notes with the limitations of each dataset.

Contents

Set	Question	Topic	Page
Set 1	Q1	Product-category BST	5
Set 1	Q2	Chaining vs built-in hash map	6
Set 1	Q3	Deployment dependencies	7
Set 1	Q4	24-hour traffic window	8
Set 1	Q5	Gift-card pair sum	9
Set 2	Q1	Claim-ID BST	11
Set 2	Q2	Normalized duplicate claims	12
Set 2	Q3	Approval-stage graph	13
Set 2	Q4	Daily claim streak	14
Set 2	Q5	Merge office claim lists	15
Set 3	Q1	Hub-code BST	17
Set 3	Q2	Repeated hub scans	18
Set 3	Q3	Ten-hub route graph	19
Set 3	Q4	Four-hour workload window	21
Set 3	Q5	Route palindrome	22

Set overview pages: Set 1 p. 3; Set 2 p. 10; Set 3 p. 16. Real Olist sample p. 4. Decision table p. 23. Sources and data limits p. 24.

Set 1 – E-Commerce Catalog and Traffic Analysis

Binary search trees · hashing · dependency graphs · sliding windows · two pointers

Business Scenario	An e-commerce platform wants faster category browsing, instant SKU look-ups, and a way to size infrastructure for peak traffic.
Tasks	**1.** Build a Binary Search Tree of product categories (insert, search) and print the inorder, preorder and postorder traversals for 15 sample categories. **2.** Implement a small hash table with chaining for SKU look-up. Demonstrate one collision, then solve the same problem with the built-in hash map and compare the look-up time for 100,000 SKUs. **3.** Model microservice deployment dependencies (cart → pricing → inventory and so on) as a graph; produce a valid deployment order with BFS and detect a circular dependency. **4.** Given requests-per-minute for 24 hours, find the busiest continuous 3-hour window using a sliding window. **5.** Given a sorted list of order values, use two pointers to find all pairs that add up to a gift-card value.
Deliverables	Source code, execution output, and a decision table listing the problem, data structure/pattern used, and the reason.

Set 1 data – ten real Olist orders

The ten orders below are actual rows from the public Olist Brazilian e-commerce data, prepared by joining order, customer, and order-item records. Amount = sum of item price + freight for the order, in BRL; the purchase timestamp is retained. Repeated keys in a worked example are matching values, not proof of fraudulent or duplicate submissions.

#	orderId	customerId	amount (BRL)	createdAt
1	246b14cc444e8a7df0afd80495dfa052	031e19fc630c4121f1238716f41675c3	R$17.97	2018-07-04 01:20:01
2	53cdb2fc8bc7dce0b6741e2150273451	af07308b275d755c9edb36a90c618231	R$141.46	2018-07-24 20:41:37
3	f014aa85d8f495d824d120e9035faacb	050555c26479bbb8ba0740c37e7454d0	R$49.44	2018-04-07 12:07:04
4	47770eb9100c2d0c44946d9cf07ec65d	3a653a41f6f9fc3d2a113cf8398680e8	R$179.12	2018-08-08 08:38:49
5	f707ff2a9514dce20b267136b60af042	031e19fc630c4121f1238716f41675c3	R$17.97	2018-07-04 01:20:01
6	4f173306347cca2ee2a05590d617854a	050555c26479bbb8ba0740c37e7454d0	R$49.44	2018-04-07 12:07:04
7	56ae029ed8bd31758f66831a64bf5629	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:10

#	orderId	customerId	amount (BRL)	createdAt
8	d9e44c3fd2ce16086619f299e92e12d8	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:10
9	84f5e6c0a0e3155e38c00f434ba90ce8	06a52782a04f0086d16b9c22d0e29438	R$51.11	2017-05-06 20:11:11
10	e481f51cbdc54678b7cc49136f2d6af7	7c396fd4830fd04220f754e42b4e5bff	R$38.71	2017-10-02 10:56:33

Column mapping

CSV field	Source meaning
orderId	orders.order_id — observed order identifier
customerId	customers.customer_unique_id — stable customer key
amount	Sum of items.price + items.freight_value grouped by order_id
createdAt	orders.order_purchase_timestamp — purchase timestamp

The selected ten orders include same-customer, same-total, same-minute groups that make suitable real worked inputs for pair-sum and scan-key demonstrations. Source.

SET 1 • QUESTION 1

Product-category BST

Question: Build a Binary Search Tree of product categories (insert, search) and print the inorder, preorder and postorder traversals for 15 sample categories.

Aim: Build and search a BST using 15 distinct product categories from Olist.

Method: Insert each category by alphabetical comparison. Search follows one branch at a time. Inorder traversal visits left, node, right and therefore prints sorted keys.

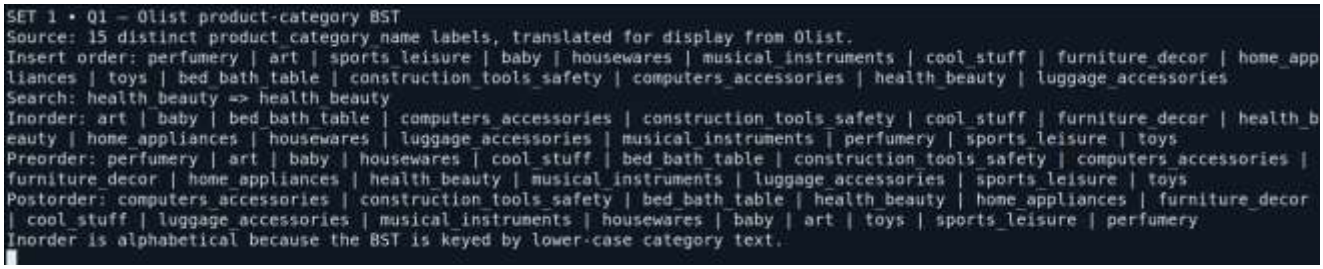
Python code (core solution)

```
class Node:
    def __init__(self, key): self.key, self.left, self.right = key, None, None
def insert(t, x):
    if t is None: return Node(x)
    if x < t.key: t.left = insert(t.left, x)
    elif x > t.key: t.right = insert(t.right, x)
    return t
def search(t, x):
    while t and t.key != x: t = t.left if x < t.key else t.right
    return t.key if t else None
def walk(t, order):
    if not t: return []
    a, b = walk(t.left, order), walk(t.right, order)
    return ([t.key]+a+b if order=='pre' else
            a+[t.key]+b if order=='in' else a+b+[t.key])
```

Dry run

Operation	Dry-run result
Insert	15 distinct translated Olist categories in product-file order
Search	health_beauty → found
Inorder	art … toys (alphabetical; complete traversal is in the terminal capture)
Preorder / postorder	All 15 nodes printed; order reflects the BST shape

Actual program output



.**Result / complexity:**

The search found health_beauty; inorder is alphabetically sorted. Average insert/search O(log n), worst O(n); traversal O(n); tree space O(n).

SET 1 • QUESTION 2

Chaining vs the built-in hash map

Question: Implement a small hash table with chaining for SKU look-up. Demonstrate one collision, then solve the same problem with the built-in hash map and compare the look-up time for 100,000 SKUs.

Aim: Show separate chaining on a deliberate collision, then compare lookup time on 100,000 real catalog IDs.

Method: Map each key to a bucket with hash(key) % m. Store colliding keys in that bucket and compare the complete key on lookup. Compare against Python dict using the same probes.

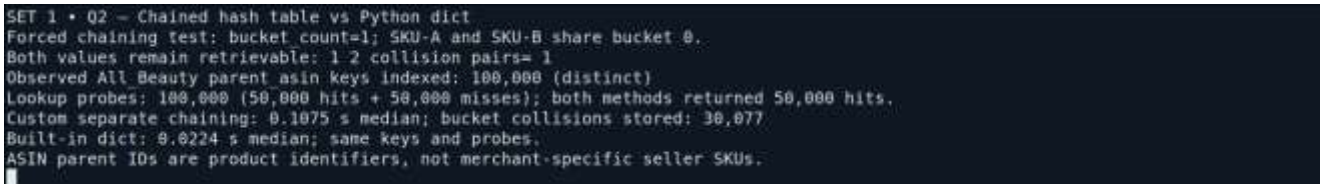
Python code (core solution)

```
from time import perf_counter
from statistics import median
class ChainedHash:
    def __init__(self, m): self.b = [[] for _ in range(m)]
    def put(self, k, v):
        bucket = self.b[hash(k) % len(self.b)]
        for i, (key, _) in enumerate(bucket):
            if key == k: bucket[i] = (k, v); return
        bucket.append((k, v))
    def get(self, k):
        for key, val in self.b[hash(k) % len(self.b)]:
            if key == k: return val
        return None
demo = ChainedHash(1); demo.put('SKU-A', 1); demo.put('SKU-B', 2)
keys = parent_asins[:100_000]; custom=ChainedHash(131071); native={}
for i,k in enumerate(keys): custom.put(k,i); native[k]=i
probes = keys[::2] + [f'NO-{i}' for i in range(50_000)]
for lookup in (custom.get, native.get):
    times=[]
    for _ in range(3):
        start=perf_counter(); hits=sum(lookup(k) is not None for k in probes)
        times.append(perf_counter()-start)
    print(hits, median(times))
```

Dry run

Test / measure	Result
Forced collision	1 bucket: SKU-A and SKU-B collide; values 1 and 2 both retrieve correctly.
Benchmark input	100,000 observed parent_asin identifiers; 100,000 lookups: 50,000 hits + 50,000 misses.
Custom chaining	0.1075 s median in this run; collision handled by comparing keys in the bucket.
Built-in dict	0.0224 s median in this run; same keys and probes.

Actual program output



Result / complexity: The built-in dict was faster in this run; timings depend on the machine and Python build. Expected lookup O(1 + α) for chaining, worst O(n); dict expected O(1). Space O(n).

SET 1 • QUESTION 3

Microservice deployment dependencies

Question: Model microservice deployment dependencies (cart → pricing → inventory and so on) as a graph; produce a valid deployment order with BFS and detect a circular dependency.

Aim: Produce a valid order for the given service dependency chain and detect a circular dependency.

Method: Treat "A depends on B" as B → A for deployment. Kahn's algorithm uses a FIFO queue (BFS-style) of services whose prerequisites are met. If not all nodes are emitted, a cycle remains.

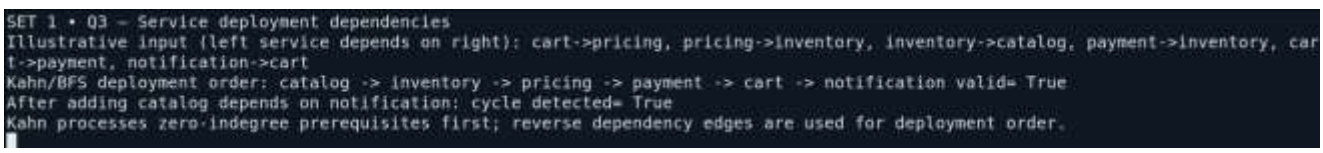
Python code (core solution)

```
from collections import deque
# A service points to its prerequisite; reverse edges for deploy order.
deps = {'cart':['pricing','payment'], 'pricing':['inventory'],
        'inventory':['catalog'], 'payment':['inventory'], 'notification':['cart']}
nodes = set(deps) | {p for v in deps.values() for p in v}
adj = {u:[] for u in nodes}; indeg = {u:0 for u in nodes}
for service, prereqs in deps.items():
    for pre in prereqs: adj[pre].append(service); indeg[service]+=1
q = deque(u for u in nodes if indeg[u]==0); order=[]
while q:
    u=q.popleft(); order.append(u)
    for v in adj[u]:
        indeg[v]-=1
        if indeg[v]==0: q.append(v)
cycle = len(order) != len(nodes)
```

Dry run

Step	Dry-run result
Dependency chain	cart → pricing → inventory → catalog
BFS / Kahn order	catalog → inventory → pricing → payment → cart → notification
Validity check	All six services emitted: valid topological deployment order.
Cycle test	Add "catalog depends on notification"; fewer than six nodes emitted → cycle.

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity: Valid order: catalog, inventory, pricing, payment, cart, notification; the added reverse dependency is detected as a cycle. O(V + E) time and O(V + E) space.

SET 1 • QUESTION 4

Busiest 3-hour traffic window

Question: Given requests-per-minute for 24 hours, find the busiest continuous 3-hour window using a sliding window.

Aim: Find the continuous three-hour period with the most requests from a real 24-hour request trace.

Method: Sum the first 180 minute bins. Slide by one minute: add the new right-edge count and subtract the dropped left-edge count. Keep the maximum.

Python code (core solution)

```
def busiest(rpm, minutes=180):
    window = sum(rpm[:minutes]); best = window; start = 0
    for right in range(minutes, len(rpm)):
        window += rpm[right] - rpm[right-minutes]
        if window > best: best, start = window, right-minutes+1
    return start, best
# rpm has 1,440 one-minute counts; 180 bins = 3 hours.
```

Dry run

Window (local -0400)	Requests in 180 minutes	Check
Bin input	1,440 real per-minute HTTP request counts for 1995-07-01; window width = 180 minutes.	NASA KSC, -0400
15:42–18:42	10454	candidate
15:43–18:43	10457	candidate
15:44–18:44	10434	candidate

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

Peak 15:43–18:43: 10,457 requests; 58.09 requests/minute on average. O(M) time for M minute bins; O(1) extra space after counts are loaded.

SET 1 • QUESTION 5

Gift-card pair sum

Question: Given a sorted list of order values, use two pointers to find all pairs that add up to a gift-card value.

Aim: Use two pointers on the ten real Olist order totals, already sorted, to find every index-pair combination for R$67.41.

Method: If the sum is too low, advance the left pointer; if too high, move the right pointer left. On a match, count repeated values so all positional pairs are included.

Python code (core solution)

```
def pair_sum(a, target):
    i, j, out = 0, len(a)-1, []
    while i < j:
        total = a[i] + a[j]
        if total < target: i += 1
        elif total > target: j -= 1
        else:
            left, right = a[i], a[j]; li, rj = i, j
            while i <= j and a[i] == left: i += 1
            while j >= i and a[j] == right: j -= 1
            out.append((left, right, (i-li)*(rj-j)))
    return out  # value pair and number of index-pair combinations
```

Dry run

Pointer values	Dry-run decision
17.97 + 179.12	197.09 > 67.41 — Move right pointer left
17.97 + 141.46	159.43 > 67.41 — Move right pointer left
17.97 + 51.11	69.08 > 67.41 — Move right pointer left (repeat value)
17.97 + 49.44	67.41 = 67.41 — Match; duplicate counts give 4 index pairs

Actual program output

```
SET 1 • Q5 — Two-pointer gift-card pair sum
Real Olist sample order totals (BRL cents, sorted): [1797, 1797, 3871, 4944, 4944, 5111, 5111, 5111, 14146, 17912]
Target: R$67.41 = 6,741 cents
R$17.97 + R$49.44 = R$67.41; index-pair combinations=4
Pairs are counted by positions; repeated order values retain their multiplicity.
```

Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

R$17.97 + R$49.44 = R$67.41; the repeated values create 4 index-pair combinations. O(n) time on the sorted list; O(1) extra space excluding the result list.

Set 2 – Insurance Claims Processing

Binary search trees · hashing/normalization · BFS/DFS · consecutive-day windows · two-pointer merge

Business Scenario	An insurer must route claims through approval stages, detect duplicate claim submissions, and monitor claim spikes after a regional event.
Tasks	**1.** Create a BST of claim IDs; insert and search; show the three traversals and explain which traversal gives claims in sorted order. **2.** Use hashing to detect duplicate or near-duplicate claims (same policy number and incident date, differing only in letter case or spacing); explain how collisions are handled. **3.** Model claim approval stages (registration, assessment, surveyor, settlement) as a directed graph and find the shortest path between two stages using BFS; detect loops using DFS. **4.** Find the longest run of consecutive days on which claims exceed 100 using a sliding window. **5.** Using two pointers, merge two sorted claim ID lists (from two regional offices) into one sorted list without an extra sort.
Deliverables	Working code, test cases covering empty input and duplicates, and a table of complexity per solution.

SET 2 • QUESTION 1

Claim-ID BST and traversals

Question: Create a BST of claim IDs; insert and search; show the three traversals and explain which traversal gives claims in sorted order.

Aim: Insert and search actual claim IDs, then compare preorder, inorder, and postorder.

Method: Compare IDs numerically at each node. Inorder traversal visits the left subtree, root, then right subtree.

Python code (core solution)

```
class N:
    def __init__(self,k): self.k,self.l,self.r=k,None,None
def put(t,k):
    if not t: return N(k)
    if k<t.k: t.l=put(t.l,k)
    elif k>t.k: t.r=put(t.r,k)
    return t
def search(t,k):
    while t and t.k!=k: t=t.l if k<t.k else t.r
    return t.k if t else None
def walk(t,mode):
    if not t: return []
    left,right=walk(t.l,mode),walk(t.r,mode)
    if mode=='pre': return [t.k]+left+right
    if mode=='in': return left+[t.k]+right
    return left+right+[t.k]
root=None
for claim_id in claim_ids: root=put(root,int(claim_id))
print(search(root,int(claim_ids[7])))
for mode in ('pre','in','post'): print(mode,walk(root,mode))
```

Dry run

Operation	**Dry-run result**
Input	15 real NY WCB claim identifiers inserted in the API sample order.
Search	7162214 → found
Inorder	Ascending numeric claim IDs; this is the sorted traversal.
Preorder / postorder	Root-first / root-last; they show tree shape, not ascending order.

Actual program output

```
SET 2 • Q1 — NY workers' compensation claim-ID BST
Observed claim IDs, inserted in API sample order: 7162591, 7162544, 7162496, 7162402, 7162355, 7162308, 7162261, 7162214, 7162079,
 7162041, 7162003, 7161963, 7161923, 7161884, 7161846
Search: 7162214 => 7162214
Inorder: 7161846, 7161884, 7161923, 7161963, 7162003, 7162041, 7162079, 7162214, 7162261, 7162308, 7162355, 7162402, 7162496, 7162
544, 7162591
Preorder: 7162591, 7162544, 7162496, 7162402, 7162355, 7162308, 7162261, 7162214, 7162079, 7162041, 7162003, 7161963, 7161923, 716
1884, 7161846
Postorder: 7161846, 7161884, 7161923, 7161963, 7162003, 7162041, 7162079, 7162214, 7162261, 7162308, 7162355, 7162402, 7162496, 71
62544, 7162591
Inorder yields ascending numeric claim IDs; other traversals show tree shape.
```

Result / complexity:

Search succeeds; inorder prints the 15 IDs in ascending order. Descending insertion creates a one-sided tree. Insert/search O(h): average O(log n), worst O(n); each traversal O(n); space O(n).

SET 2 • QUESTION 2

Normalized duplicate-claim keys

Question: Use hashing to detect duplicate or near-duplicate claims (same policy number and incident date, differing only in letter case or spacing); explain how collisions are handled.

Aim: Normalize case and spacing in a policy number, pair it with incident date, and report repeated keys.

Method: Use a hash map keyed by (normalized policy number, incident date). A hash collision only means two keys share a bucket; equality is confirmed by comparing the full tuple.

Python code (core solution)

```
import re
from collections import defaultdict
def norm(s): return re.sub(r"\s+", "", s).upper()
def duplicate_groups(rows):
    groups=defaultdict(list)
    for r in rows:
        key=(norm(r["policy_number"]), r["incident_date"])
        groups[key].append(r)
    return {k:v for k,v in groups.items() if len(v)>1}
# Hash selects a bucket; compare the full normalized tuple inside it.
```

Dry run

Case	Policy number	Incident date / outcome	Key / test result
Test row	Policy input	Date	Normalized key
1	AB 120-7	2019-06-03	AB120-7 | 2019-06-03
2	ab120-7	2019-06-03	AB120-7 | 2019-06-03 → same group
3	AB120-7	2019-06-04	AB120-7 | 2019-06-04 → separate group

Case	Policy number	Incident date / outcome	Key / test result
Edge tests	Empty input → no groups; forced one-bucket collision → 2 distinct full keys retained.		

Actual program output

```
SET 2 • Q2 — Normalized duplicate-claim key
Test-only rows; public real claim data omits individual policy numbers.
policy='AB 120-7'; incident_date=2019-06-03; key=AB1207|2019-06-03
policy=' ab120-7 '; incident_date=2019-06-03; key=AB1207|2019-06-03
policy='AB120-7'; incident_date=2019-06-04; key=AB1207|2019-06-04
Same normalized policy + date group sizes: [2]
Empty claims input => duplicate groups = 0.
Forced one-bucket collision: distinct complete keys retained = 2
Hash collisions are resolved by comparing the full normalized key, not by assuming equal hashes mean duplicates.
```

Result / complexity:

Two case/spacing variants form one candidate group; the same policy on a different incident date

SET 2 • QUESTION 3

Approval-stage graph: BFS and DFS

Question: Model claim approval stages (registration, assessment, surveyor, settlement) as a directed graph and find the shortest path between two stages using BFS; detect loops using DFS.

Aim: Find the fewest stage transitions from registration to settlement and detect a workflow loop.

Method: BFS records each stage's parent and returns the shortest unweighted path. DFS marks visiting/visited states; an edge to a currently visiting node closes a cycle.

Python code (core solution)

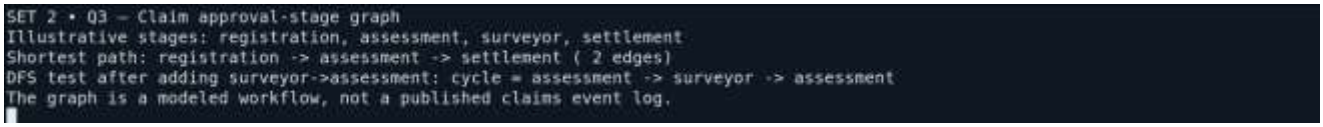
```
from collections import deque
def shortest(adj,s,t):
    q=deque([s]); parent={s:None}
    while q:
        u=q.popleft()
        if u==t: break
        for v in adj[u]:
            if v not in parent: parent[v]=u; q.append(v)
    if t not in parent: return None
    path=[]; u=t
    while u is not None: path.append(u); u=parent[u]
    return path[::-1]
def has_cycle(adj,nodes):
    color={u:0 for u in nodes}
    def visit(u):
        color[u]=1
```

```
        for v in adj[u]:
            if color[v]==1: return True
            if color[v]==0 and visit(v): return True
        color[u]=2; return False
    return any(color[u]==0 and visit(u) for u in nodes)
```

Dry run

Stage / test	Outgoing edge or result
registration	assessment
assessment	surveyor, settlement
surveyor	settlement
settlement	(end)
BFS path	registration → assessment → settlement (2 edges)
DFS loop test	Add surveyor → assessment; back-edge reveals assessment → surveyor → assessment.

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

Shortest route is registration → assessment → settlement. The added reverse edge makes a cycle. BFS and DFS each O(V + E) time; O(V + E) graph space and O(V) traversal space.

SET 2 • QUESTION 4

Consecutive high-claim days

Question: Find the longest run of consecutive days on which claims exceed 100 using a sliding window.

Aim: Count distinct claims assembled each calendar day; find the longest consecutive streak strictly above 100.

Method: This is a variable-length rolling streak: extend the current window when count > 100 on the next calendar day; reset on a low or missing day. Use assembly_date, not accident_date.

Python code (core solution)

```
from datetime import date
def longest_run(daily):
    best=cur=0; start=best_start=best_end=None; prev=None
    for day,count in daily:
        day=date.fromisoformat(day) if isinstance(day,str) else day
        if count>100:
            if cur==0 or prev is None or (day-prev).days!=1: start=day; cur=0
            cur+=1
            if cur>best: best,best_start,best_end=cur,start,day
        else: cur=0; start=None
        prev=day
    return best,best_start,best_end
```

Dry run

Date	Claims	Date	Claims
2019-06-01	0	2019-06-09	129
2019-06-02	0	2019-06-10	1012
2019-06-03	1095	2019-06-11	1178
2019-06-04	1161	2019-06-12	1224
2019-06-05	1287	2019-06-13	1262
2019-06-06	1329	2019-06-14	1250
2019-06-07	236	2019-06-15	1
2019-06-08	1128	2019-06-16	0

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

Longest run: 12 calendar days, 2019-06-03 through 2019-06-14. Missing dates are treated as zero. O(d) time and O(1) extra space after daily aggregation, where d is number of dates. [4]

SET 2 • QUESTION 5

Merge two district-office claim lists

Question: Using two pointers, merge two sorted claim ID lists (from two regional offices) into one sorted list without an extra sort.

Aim: Merge sorted claim identifiers from two real New York WCB district offices without sorting again.

Method: Compare the current IDs, emit the smaller one, and advance only that list's pointer. When one list ends, append the other remainder.

Python code (core solution)

```
def merge(a,b):
    i=j=0; out=[]
    while i<len(a) and j<len(b):
        if int(a[i])<=int(b[j]): out.append(a[i]); i+=1
        else: out.append(b[j]); j+=1
    return out+a[i:]+b[j:]
# No sort call: advance only the pointer for the smaller current ID.
```

Dry run

Step	Dry-run result
Input A	Buffalo: 7162048, 7162080, 7162081, 7162082, 7162083, 7162084, 7162085, 7162086
Input B	Rochester: 7162069, 7162101, 7162102, 7162103, 7162104, 7162105, 7162106, 7162107
Compare 1	7162048 < 7162069 → emit Buffalo 7162048
Compare 2	7162080 > 7162069 → emit Rochester 7162069
Remainder	Continue comparing; append remaining sorted IDs.
Checks	16 IDs sorted; merge([], first two Buffalo IDs) returns those two IDs.

Actual program output

```
SET 2 • Q5 – Merge actual NY WCB district-office claim lists
Buffalo IDs sorted: 7162048, 7162080, 7162081, 7162082, 7162083, 7162084, 7162085, 7162086
Rochester IDs sorted: 7162069, 7162101, 7162102, 7162103, 7162104, 7162105, 7162106, 7162107
Merged ascending without calling sort(): 7162048, 7162069, 7162080, 7162081, 7162082, 7162083, 7162084, 7162085, 7162086, 7162101,
 7162102, 7162103, 7162104, 7162105, 7162106, 7162107
Sorted and contains all IDs: True count= 16
Empty-list test: ['7162048', '7162080']
```

Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

The Buffalo and Rochester records merge into one ascending list of 16 IDs; an empty-list edge test also passes. O(m + n) time; O(m + n) output space (or O(1) auxiliary space if output is written in place).

Set 3 – Logistics Network and Delivery Windows

Binary search trees · hash maps · adjacency lists · BFS/DFS · sliding windows · two pointers

Business Scenario	A courier company is optimising hub-to-hub routing and delivery-window planning across a national network.
Tasks	**1.** Store hub codes in a BST and implement search and insert; identify the tree height for the given insertion order and explain the effect of sorted input. **2.** Build a lookup for parcel tracking numbers using a hash map; implement a check that reports parcels scanned twice at the same hub. **3.** Represent 10 hubs and direct routes as an adjacency list; find the route with the fewest hops between two hubs (BFS) and list every hub reachable from the central depot (DFS). **4.** Given hourly parcel counts, find the 4-hour window with the highest average using a sliding window. **5.** Use two pointers to check whether a delivery route description is a palindrome after ignoring case and separators.
Deliverables	Code, the graph diagram or adjacency list used, output, and a short explanation of every algorithm choice.

SET 3 • QUESTION 1

Hub-code BST and tree height

Question: Store hub codes in a BST and implement search and insert; identify the tree height for the given insertion order and explain the effect of sorted input.

Aim: Use ten observed OpenFlights airport codes; measure height for the stated insertion order and sorted order.

Method: Insert/search compare airport-code strings lexicographically. Height is the number of nodes on the longest root-to-leaf path.

Python code (core solution)

```
class N:
   def __init__(self,k): self.k,self.l,self.r=k,None,None
def insert(t,k):
   if not t: return N(k)
   if k<t.k: t.l=insert(t.l,k)
   elif k>t.k: t.r=insert(t.r,k)
   return t
def search(t,k):
   while t and t.k!=k: t=t.l if k<t.k else t.r
   return t.k if t else None
def height(t):
   return 0 if not t else 1+max(height(t.l),height(t.r))
root=None
for hub in hubs: root=insert(root,hub)
print(search(root,'AAN'),height(root))
```

Dry run

Operation	Dry-run result
Input order	AAE, ALG, DOH, CCJ, AAN, CDG, IST, LYS, MRS, ORN
Search	AAN → found
Measured height	7 levels in this insertion order
Sorted input	10 levels for 10 nodes: a degenerate left/right chain.

Actual program output

```
SET 3 • Q1 — BST of real OpenFlights hub codes
Insertion order: AAE, ALG, DOH, CCJ, AAN, CDG, IST, LYS, MRS, ORN
Search AAN: AAN
Inorder: AAE, AAN, ALG, CCJ, CDG, DOH, IST, LYS, MRS, ORN
Height in stated order: 7 levels; sorted-order height: 10 levels for 10 nodes.
Height counts nodes on the longest root-to-leaf path; sorted insertion creates a chain.
```

Result / complexity:

Height = 7 for the selected order; inserting all ten codes sorted gives height 10 and O(n) worst-case search. Insert/search O(h), worst O(n); height calculation O(n); tree space O(n).

SET 3 • QUESTION 2

Repeated parcel scan at the same hub

Question: Build a lookup for parcel tracking numbers using a hash map; implement a check that reports parcels scanned twice at the same hub.

Aim: Report a repeated pair of tracking ID and hub code while allowing the same parcel to be scanned at a different hub.

Method: A hash map stores a set of hubs per tracking ID. A repeated (tracking ID, hub) pair is flagged; a new hub for the same tracking ID is not. IDs are shortened in the table; the capture shows full IDs.

Data limit: Do not treat these event rows as observed delivery scans; the screenshot shows a test fixture using real identifiers.

Python code (core solution)

```
from collections import defaultdict
def repeated_scans(events):
    seen=defaultdict(set); duplicates=[]
    for tracking_id, hub in events:
        if hub in seen[tracking_id]: duplicates.append((tracking_id,hub))
        else: seen[tracking_id].add(hub)
    return duplicates
# Key by tracking ID; set membership checks the same hub.
```

Dry run

Scan #	Tracking ID	Hub	Decision
1	246b14cc…dfa052	AAE	new package/hub key
2	53cdb2fc…273451	DOH	new package/hub key
3	246b14cc…dfa052	AAE	duplicate same-hub scan
4	246b14cc…dfa052	ALG	same package, different hub

Actual program output

```
SET 3 • Q2 — Parcel scan hash map
TEST FIXTURE: real Olist order IDs and real airport codes; scan events are illustrative, not source logs.
scan 246b14cc444e8a7df0afd80495dfa052 at AAE
scan 53cdb2fc8bc7dce0b6741e2150273451 at DOH
scan 246b14cc444e8a7df0afd80495dfa052 at AAE
scan 246b14cc444e8a7df0afd80495dfa052 at ALG
Repeated (tracking ID, same hub) pairs: [('246b14cc444e8a7df0afd80495dfa052', 'AAE')]
A repeat at a different hub is not counted as a duplicate same-hub scan.
```

Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

The repeated AAE scan is detected; the later ALG scan for the same ID is a different hub. Expected O(e) time and O(u) space for e events and u unique package–hub pairs.

SET 3 • QUESTION 3

Ten-hub adjacency list, BFS and DFS

Question: Represent 10 hubs and direct routes as an adjacency list; find the route with the fewest hops between two hubs (BFS) and list every hub reachable from the central depot (DFS).

Aim: Build the induced graph for ten real airport codes; find a fewest-hop path and list every hub reachable from AAE.

Method: Use an adjacency list from observed direct OpenFlights routes. BFS returns a shortest unweighted path; DFS with a visited set enumerates reachable nodes.

Data limit: The ten nodes are real airport codes; the direct edges are real historical airline routes. They should not be interpreted as parcel routes.

Python code

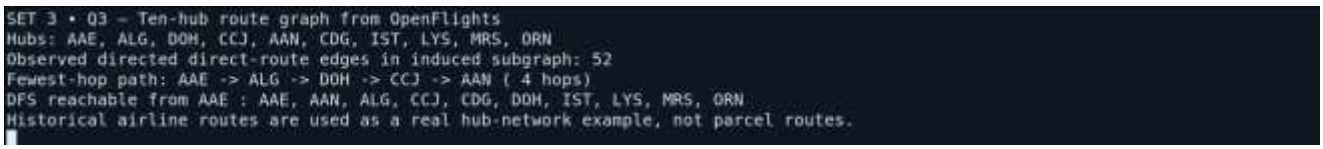
```
from collections import deque
def shortest(adj,s,t):
    q=deque([s]); parent={s:None}
    while q:
        u=q.popleft()
        for v in adj[u]:
            if v not in parent: parent[v]=u; q.append(v)
    if t not in parent: return None
    path=[]; x=t
    while x is not None: path.append(x); x=parent[x]
    return path[::-1]
def reachable(adj,start):
    seen=set(); stack=[start]
    while stack:
        u=stack.pop()
        if u not in seen: seen.add(u); stack.extend(adj[u])
    return seen
adj={hub:[] for hub in hubs}
for origin,destination in routes: adj[origin].append(destination)
print(shortest(adj,'AAE','AAN'))
print(sorted(reachable(adj,'AAE')))
```

Dry run

Hub	Observed direct-route neighbors in the ten-hub sample
AAE	ALG, CDG, IST, LYS, MRS, ORN
ALG	AAE, CDG, DOH, IST, LYS, MRS, ORN
DOH	ALG, CCJ, CDG, IST
CCJ	AAN, DOH
AAN	CCJ

Hub	Observed direct-route neighbors in the ten-hub sample
CDG	AAE, ALG, DOH, IST, LYS, MRS, ORN
IST	AAE, ALG, CDG, DOH, LYS, MRS, ORN
LYS	AAE, ALG, CDG, IST, MRS, ORN
MRS	AAE, ALG, CDG, IST, LYS, ORN
ORN	AAE, ALG, CDG, IST, LYS, MRS

Actual program output



Result / complexity:

BFS path AAE → ALG → DOH → CCJ → AAN (4 hops); DFS reaches all ten listed hubs. BFS/DFS O(V + E) time; adjacency-list storage O(V + E).

SET 3 • QUESTION 4

Four-hour delivery workload window

Question: Given hourly parcel counts, find the 4-hour window with the highest average using a sliding window.

Aim: Find the four consecutive hours with the highest average from 24 real hourly Olist order-arrival counts.

Method: Start with the first four-hour total. Slide one hour at a time by adding the incoming count and subtracting the outgoing count; divide each sum by four.

Data limit: The public source has orders and purchase timestamps, not hourly parcel scan counts. This is an explicit workload proxy.

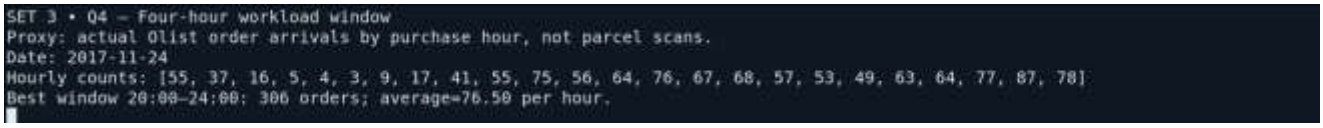
Python code (core solution)

```
def best_window(counts,k=4):
    total=sum(counts[:k]); best=total; start=0
    for right in range(k,len(counts)):
        total += counts[right]-counts[right-k]
        if total>best: best,start=total,right-k+1
    return start,best/k,best
# 24 actual order-arrival counts; k=4 consecutive hours.
```

Dry run

Window / date	Total orders	Average / note	Input
Observed date	2017-11-24	24 hourly bins	Olist order purchase times
19:00–23:00	291	average 72.75/hour	candidate
20:00–24:00	306	average 76.50/hour	candidate

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity: Best window 20:00–24:00 on 2017-11-24: 306 orders, averaging 76.50 per hour. O(H) time and O(1) extra space for H hourly bins.

SET 3 • QUESTION 5

Route-description palindrome test

Question: Use two pointers to check whether a delivery route description is a palindrome after ignoring case and separators.

Aim: Ignore case and separators, then compare characters from both ends using two pointers.

Method: Normalize to alphanumeric characters. Compare left and right; return false at the first mismatch, otherwise move inward until the pointers meet.

Data limit: No public source was used to claim either string is an observed parcel route.

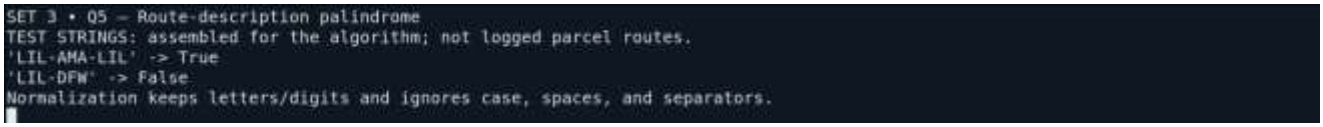
Python code (core solution)

```
def palindrome(text):
    s=''.join(c.casefold() for c in text if c.isalnum())
    i,j=0,len(s)-1
    while i<j:
        if s[i]!=s[j]: return False
        i+=1; j-=1
    return True
# Punctuation, spaces, and separators are ignored.
```

Dry run

Route text	Normalized input	Dry-run check	Result
Input	Normalized text	Pointer comparison	Result
LIL-AMA-LIL	LILAMALIL	L=L, I=I, L=L, A=A	Palindrome
LIL-DFW	LILDFW	L ≠ W at first comparison	Not a palindrome

Actual program output



Terminal capture from the actual Python run on the cited or labeled input.

Result / complexity:

LIL-AMA-LIL passes; LIL-DFW fails. The strings are constructed tests using real airport codes, not logged routes. O(L) time and O(L) extra space for the normalized string; an in-place scan can reduce auxiliary space to O(1).

Decision and complexity table

Question	Pattern / structure	Problem	Time / space	Reason
S1 Q1	BST	Category lookup and ordered traversal	Search/insert avg O(log n), worst O(n); traversal O(n)	Keeps keys ordered; inorder produces sorted categories.
S1 Q2	Chained hash map / dict	Fast catalog key lookup	Expected O(1+α); worst O(n)	Separate chaining preserves colliding keys; dict is the practical baseline.
S1 Q3	Directed graph + Kahn queue	Dependency order and cycle test	O(V+E)	Prerequisites become zero-indegree services; incomplete output signals a cycle.
S1 Q4	Fixed sliding window	Peak 180-minute traffic total	O(M), O(1) extra	Only outgoing and incoming minute counts change.
S1 Q5	Two pointers	Find all value pairs in a sorted list	O(n), O(1) extra	Sorted order lets a low/high sum guide one pointer at a time.
S2 Q1	BST	Insert/search claim IDs; traversals	O(h); worst O(n)	Inorder visits numeric IDs in ascending order.
S2 Q2	Hash map + normalized tuple	Near-duplicate candidate grouping	Expected O(n); worst $O(n^2)$	Normalize case/spacing; compare full tuple to separate hash collisions.
S2 Q3	Adjacency list + BFS/DFS	Fewest stages and cycle detection	O(V+E)	BFS minimizes unweighted hops; DFS back-edge detects a loop.
S2 Q4	Variable-length rolling streak	Longest run with daily count >100	O(d), O(1) extra	Extend on consecutive high days; reset on low/missing day.
S2 Q5	Two-pointer merge	Merge sorted office claim IDs	O(m+n)	Emit smaller current ID; no extra sort.
S3 Q1	BST	Hub search/insert and height	O(h); worst O(n)	Sorted insertion can form a chain of height n.
S3 Q2	Hash map of hub sets	Repeated package scan at same hub	Expected O(e)	Key by tracking ID and check whether hub was already seen.
S3 Q3	Adjacency list + BFS/DFS	Fewest hops and reachability	O(V+E)	BFS finds shortest unweighted path; DFS marks reachable hubs.
S3 Q4	Four-hour sliding window	Highest average workload block	O(H), O(1) extra	Slide totals without recomputing all four hours.
S3 Q5	Two pointers	Palindrome after normalization	O(L), O(L) normalized copy	Compare from both ends and stop at the first mismatch.